OFFLINE-FIRST SECURITY ENGINEERING

VNAGY Coding Standard Specification

Internal Technical Standard for Minimal, Deterministic, and Explicit Code Practices

Document ID: VNAGY-SE-CS-STD-001

Version: v1.2 (Structural & Content Expansion Update)

Revision History:

Version	Date	Author	Description
V1.0	20 July 2026	V. Nad	Draft
V1.1	22 July 2026	V. Nad	License & Footer Integrated
V1.2	01 August 2026	V. Nad	Structural & Content Expansion Update

Document Status: Internal Technical Specification

Prepared by: VNAGY Security Engineering

Reviewed by: V. Nađ — Internal Peer Review

Approved by: V. Nađ — Standards Committee

Location: Osijek

Contact: viktorijanad8@gmail.com

I. Executive Summary

The **VNAGY** Coding Standard defines a minimal, deterministic, and explicit set of rules for writing, structuring, and maintaining code in offline-first security-oriented environments. The standard ensures that all system components follow a unified logic, predictable execution flow, and strictly controlled dependencies, resulting in high stability, security, and long-term maintainability.

The primary objective of this document is to establish a consistent technical framework that enables straightforward verification, clear readability, and a minimal surface for errors. The rules defined in this standard apply to all parts of the system, including architecture, implementation, documentation, testing, and licensing.

This standard is intended for engineers, technical authors, and contributors working on system components, and requires adherence to all normative provisions outlined in the document. Non-normative sections serve as guidance for contextual understanding but do not represent mandatory requirements.

II. Abstract

This document defines the official **VNAGY** coding standard for minimal, deterministic, and explicit software development practices across all VNAGY Security Engineering projects. It establishes mandatory rules for code structure, naming, formatting, documentation, and architectural consistency to ensure reliability, offline-first resilience, and long-term maintainability. The specification provides a unified baseline for all contributors, enabling predictable behavior, reduced ambiguity, and strict alignment with **VNAGY's** security-driven engineering principles.

III. Scope Limitations

This document defines the structural, stylistic, and normative rules required for producing deterministic, explicit, and minimal technical artifacts within the **VNAGY** ecosystem. The scope of this standard is intentionally limited to documentation, coding conventions, architectural principles, and non-operational guidelines that ensure consistency and predictability across all components.

The document does **not** describe internal algorithms, operational logic, security heuristics, behavioral pipelines, scoring mechanisms, or any implementation-specific details. These elements remain strictly internal and are outside the scope of this publication.

The standard does **not** cover deployment procedures, runtime behavior, infrastructure configuration, or system-level execution flows. Any operational or environment-dependent instructions are explicitly excluded.

This document applies only to the structural and normative aspects of **VNAGY** and must not be interpreted as a functional specification, implementation manual, or security operations guide.

IV. Scope

This specification applies to all **VNAGY Security Engineering software projects**, modules, libraries, and internal tools. It defines mandatory coding rules, formatting requirements, naming conventions, documentation standards, and structural guidelines that must be followed by all contributors. The scope includes source code, configuration files, internal scripts, and supporting development assets. The specification does not cover product design, UI/UX guidelines, infrastructure deployment, or operational security procedures.

V. Definitions

V. I. Deterministic Code

Code that produces the same output for the same input without hidden or implicit state. Determinism ensures predictability, testability, and offline-first reliability.

V. II. Explicit State

State that is fully visible, controlled, documented, and intentionally managed. No hidden transitions or implicit mutations are allowed.

V. III. Minimal Surface

The smallest possible public API or module interface required for functionality. Reduces complexity, attack surface, and maintenance overhead.

V. IV. Offline-First

A design principle where software must operate without network connectivity, ensuring resilience, determinism, and security regardless of external dependencies.

V. V. Module

A self-contained, deterministic unit of functionality with explicit inputs, outputs, and documentation. Modules must follow **VNAGY** structural and naming rules.

V. VI. Component

A higher-level construct composed of multiple modules. Components must maintain deterministic behavior and minimal coupling.

V. VII. Contributor

Any individual who writes, modifies, or maintains **VNAGY** code, documentation, or development assets.

V. VIII. Internal Tool

A **VNAGY**-maintained script, utility, or automation asset used to support development, testing, or deployment workflows.

V. IX. Coding Standard

A mandatory set of rules governing code structure, naming, formatting, documentation, and deterministic behavior across all **VNAGY** projects.

V. X. Security Engineering

The discipline focused on designing and implementing systems that maintain predictable, secure behavior under all conditions, including offline operation.

V. XI. Configuration Artifact

Any file or asset that defines system behavior, module parameters, or environment settings. Must follow explicit formatting and documentation rules.

V. XII. Development Asset

Supporting files used during development, such as scripts, templates, or metadata. Must adhere to **VNAGY** structural and naming conventions.

VI. Normative References

The following documents are referenced as authoritative sources and form the normative foundation for this specification:

- **VNAGY Architecture Specification** — defines structural, modular, and offline-first design principles used across all VNAGY systems.
- **VNAGY Security Principles** — establishes mandatory security requirements and deterministic behavior rules.
- **VNAGY Offline-First Model** — outlines constraints and guarantees for systems operating without network dependency.
- I**SO/IEC 27001** — international standard for information security management systems.
- **NIST SP 800-63** — digital identity guidelines providing structure for deterministic and secure authentication flows.
- **Microsoft SDL Coding Guidelines** — secure development lifecycle rules influencing VNAGY coding and documentation practices.

VII. Compliance Requirements

The following requirements define mandatory and recommended rules that all contributors must follow when writing, modifying, or maintaining **VNAGY** software components. These compliance statements ensure deterministic behavior, explicit state management, minimalism, and offline-first reliability across all **VNAGY** systems.

VII. I. Mandatory Requirements

Code **MUST** be deterministic and produce identical results for identical inputs.

- All state **MUST** be explicit, documented, and intentionally controlled.
- Modules **MUST** follow **VNAGY** structural and naming conventions.
- Public interfaces **MUST** expose a minimal surface.
- Error handling **MUST** be explicit, predictable, and non-implicit.
- All configuration artifacts **MUST** follow **VNAGY** formatting and documentation rules.
- All code **MUST** be compatible with offline-first operation.
- Security-relevant logic **MUST** avoid dynamic or implicit behavior.

VII. II. Prohibited Requirements

- Code **MUST NOT** introduce hidden state or implicit mutations.
- Modules **MUST NOT** depend on unpredictable external systems (e.g., network availability).
- Components **MUST NOT** expose unnecessary public functions or data.
- Error handling **MUST NOT** rely on silent failures or implicit fallbacks.
- Contributors **MUST NOT** bypass **VNAGY** documentation or formatting rules.

VII. III. Recommended Requirements

- Modules **SHOULD** minimize dependencies and coupling.
- Components **SHOULD** maintain clear separation of concerns.

- Documentation **SHOULD** be concise, minimal, and placed close to the code it describes.
- Contributors **SHOULD** use **VNAGY** internal tools for validation, formatting, and consistency checks.

VII. IV. Optional Requirements

- Components **MAY** extend functionality if extensions remain deterministic and minimal.
- Modules **MAY** include internal helper functions if they do not expand the public surface.
- Contributors **MAY** propose new coding rules if they align with **VNAGY** principles.

VIII. Coding Principles

VIII. I. Minimalism

Code must expose the smallest possible interface, contain only essential logic, and avoid unnecessary abstractions. Minimalism reduces complexity, attack surface, and maintenance overhead.

VIII. II. Determinism

All code must behave predictably and produce identical results for identical inputs. Deterministic execution is mandatory for offline-first reliability, testing, and security guarantees.

VIII. III. Explicitness

State, behavior, dependencies, and data flows must be fully visible and intentionally controlled. No implicit mutations, hidden transitions, or ambiguous logic are allowed.

VIII. IV. Isolation

Modules must be self-contained, independent, and free of unnecessary coupling. Each module must define clear inputs, outputs, and responsibilities.

VIII. V. Predictability

Code must avoid dynamic behavior, runtime surprises, and implicit fallbacks. Predictable execution ensures stable operation in constrained or offline environments.

VIII. VI. Consistency

Naming, formatting, structure, and documentation must follow **VNAGY** standards uniformly across all projects. Consistency enables readability, maintainability, and cross-team collaboration.

VIII. VII. Offline-First Reliability

All code must operate correctly without network access. External dependencies must be optional, controlled, and never required for core functionality.

VIII. VIII. Security by Design

Code must follow safe defaults, avoid unnecessary exposure, and minimize trust assumptions. Security is embedded into every module, not added later.

IX. Code Structure

VNAGY codebases must follow a predictable, minimal, and deterministic structure. All modules, components, and development assets must be organized in a way that ensures clarity, isolation, and offline-first reliability. The following structural rules apply to all **VNAGY** software projects.

IX. I. Project Root Layout

Every **VNAGY** project **MUST** follow the standard root layout:

- **src** — contains all production modules and components
- **config** — explicit configuration artifacts
- **docs** — internal documentation, specifications, and module descriptions
- **tests** — deterministic test suites
- **integration** — integration logic, adapters, or bridges
- **tools** — internal scripts and development assets
- **profiles** — environment-specific offline-first profiles (optional)

This structure ensures consistency across all **VNAGY** repositories.

IX. II. Module Structure

Each module **MUST** be self-contained and follow this internal layout:

- **module_name/**
    - **main.vng** — primary deterministic logic
    - **state.vng** — explicit state definitions
    - **errors.vng** — explicit error types
    - **types.vng** — data structures and interfaces
    - **README.md** — minimal documentation
    - **tests/** — module-specific deterministic tests

Modules **MUST NOT** expose unnecessary files or dynamic behavior.

IX. III. Component Structure

Components are composed of multiple modules and **SHOULD** follow this pattern:

- **component_name/**
    - **modules/** — isolated modules
    - **adapter/** — deterministic interfaces to external systems
    - **offline/** — offline-first logic and fallbacks
    - **config/** — component-specific configuration artifacts
    - **README.md**

Components **MUST** maintain minimal coupling and explicit boundaries.

IX. IV. File Naming Rules

- File names **MUST** be lowercase with hyphens: `module-name.vng`
- Directories **MUST** use lowercase: `module_name/`
- No spaces, no camelCase, no dynamic naming
- Public files **MUST** reflect their purpose:
    - **`errors.vng`**
    - **`state.vng`**
    - **`types.vng`**
    - **`main.vng`**

This ensures predictability and readability.

IX. V. Deterministic Folder Behavior

- No folder **MAY** contain mixed responsibilities
- No folder **MAY** contain hidden state
- No folder **MAY** contain dynamic runtime artifacts
- All folders **MUST** be safe for offline operation

IX. VI. Separation of Concerns

- Logic, state, errors, and types **MUST** be separated
- Documentation **MUST** be placed next to the code it describes
- Tests **MUST** mirror the structure of the modules they validate

IX. VII. Prohibited Structures

The following patterns **MUST NOT** appear in **VNAGY** projects:

- `utils/` folders with mixed logic
- `helpers/` folders without explicit purpose
- deeply nested dynamic structures
- implicit module grouping
- runtime-generated directories

VNAGY architecture requires strict determinism and explicitness.

X. Naming Conventions

All names used in **VNAGY** software components **MUST** follow strict deterministic and minimal rules. Naming conventions ensure clarity, predictability, and consistency across all **VNAGY** modules, components, and configuration artifacts.

X. I. Directory Naming

- Directories **MUST** use lowercase.
- Words **MUST** be separated with hyphens or underscores.
- Directory names **MUST** reflect their purpose.

Examples:

- **src/**
- **config/**
- **tests/**
- **module-name/**
- **component_name/**

X. II. Module Naming

- Module names **MUST** be lowercase.
- Module names **MUST** be short and descriptive.
- Module names **MUST NOT** contain spaces or camelCase.
- Module names **SHOULD** reflect deterministic functionality.

Examples:

- **auth-module/**
- **device-detection/**
- **profile-loader/**

X. III. Component Naming

- Components **MUST** use lowercase.
- Components **MUST** describe the system-level responsibility.
- Components **SHOULD** avoid abbreviations unless widely understood.

Examples:

- **agent/**
- **agent-installer/**
- **ntegration-layer/**

X. IV. Type Naming

- Types **MUST** use PascalCase.
- Types **MUST** be explicit and descriptive.
- Types **MUST NOT** use abbreviations.

Examples:

- UserProfile
- DeviceState
- ErrorCode
- OfflineContext

X. V. Error Naming

- Error identifiers **MUST** use PascalCase.
- Error names **MUST** end with Error.
- Error names **MUST** describe the failure explicitly.

Examples:

- **InvalidStateError**
- **MissingConfigError**
- **OfflineModeError**

X. VI. Constant Naming

- Constants **MUST** use UPPERCASE.
- Words **MUST** be separated with underscores.
- Constant names **MUST** be deterministic and explicit.

Examples:

- **DEFAULT_TIMEOUT**
- **MAX_RETRY_COUNT**
- **OFFLINE_MODE_ENABLED**

X. VII. Function Naming

- Functions **MUST** use lowercase with hyphens or underscores.
- Function names **MUST** describe deterministic behavior.
- Function names **MUST NOT** be ambiguous.

Examples:

- **load-profile()**
- **validate-state()**
- **resolve-errors()**
- **apply-config()**

X. VIII. Variable Naming

- Variables **MUST** use lowercase with hyphens or underscores.
- Variable names **MUST** be minimal but descriptive.
- Variables **MUST NOT** use camelCase.

Examples:

- **current_state**
- **device_id**
- **profile_path**

X. IX. Prohibited Naming Patterns

The following patterns **MUST NOT** appear in VNAGY codebases:

- camelCase **(`deviceId`, `userProfile`)**
- ambiguous names **(`temp`, `data`, `info`)**
- dynamic names **(`file1`, `file2`, `moduleX`)**
- mixed-case directories **(`ModuleName/`)**
- names without purpose **(`misc/`, `helpers/`, `utils/`)**

XI. Formatting Rules

All **VNAGY** documents **MUST** follow strict formatting rules to ensure clarity, consistency, and deterministic presentation across all specifications, modules, and whitepapers. Formatting is not aesthetic; it is a structural requirement.

XI. I. Typography Rules

- Section titles (**H1**) **MUST** use **24 pt**, bold.
- Subsection titles (**H2**) **MUST** use **16 pt**, bold.
- Third-level titles (**H3**) **MUST** use **12 pt**, bold.
- Body text **MUST** use **11 pt** regular.
- Notes, examples, and footnotes **MAY** use **10 pt**.

XI. II. Alignment Rules

- All text **MUST** be left-aligned.
- Titles **MUST NOT** be centered.
- Code examples **MUST** use monospaced alignment.
- Tables **MUST** be left-aligned with consistent column width.

XI. III. Color and Highlighting

- VNAGY documents **MUST** use monochrome formatting (black/gray).
- Colors **MUST NOT** be used for emphasis.
- Bold **MAY** be used for key terms.
- Italics **MAY** be used for definitions or references.

XI. IV. Lists and Structure

- Ordered lists **MUST** use Arabic numerals (1, 2, 3).
- Unordered lists **MUST** use hyphens (-).
- Nested lists **SHOULD NOT** be used.
- Each list item **MUST** be a complete, meaningful statement.

XI. V. Tables

- Tables **MUST** have a header row.
- Columns **MUST** be left-aligned.
- Tables **MUST NOT** contain merged cells.
- Tables **MUST** be used only for structured comparisons or definitions.
- Tables **MUST** be used only for structured comparisons or definitions in core sections of the standard.
- Metadata tables (title page, revision history, appendix) **MAY** use flexible column width, variable cell spacing, and non-deterministic layout, as they represent document metadata rather than normative technical content.

XI. VI. Code Formatting

- Code blocks **MUST** use monospaced font.
- Code examples **MUST** be minimal and deterministic.
- Code **MUST NOT** include dynamic or ambiguous placeholders.
- File names **MUST** be shown exactly as used (`main.vng`, `state.vng`).

XI. VII. Document Layout

- Margins **MUST** use asymmetric layout:
    - **Left: 2.5 cm**
    - **Right: 2.0 cm**
    - **Top: 2.5 cm**
    - **Bottom: 2.0 cm**

- Page numbers **MUST** be bottom-center or bottom-right.
- Headers **MAY** include document version, section title, or classification.
- Footers **MUST NOT** contain decorative elements or non-functional graphics.

XI. VIII. Prohibited Formatting

The following formatting patterns **MUST NOT** appear in **VNAGY** documents:

- centered titles
- colored text
- decorative fonts
- inconsistent spacing
- mixed alignment
- camelCase in headings
- oversized or undersized typography
- nested lists deeper than one level

XII. Documentation Rules

All **VNAGY** documentation MUST follow strict deterministic, minimal, and explicit rules. Documentation is not decoration; it is a functional component of the **VNAGY** architecture.

XII. I. General Documentation Principles

- Documentation **MUST** be minimal, explicit, and deterministic.
- Documentation **MUST NOT** contain narrative, filler, or subjective language.
- Documentation **MUST** describe behavior, structure, and constraints.
- Documentation **MUST** be updated whenever code structure changes.
- Documentation **MUST** be stored alongside the code it describes.

XII. II. Required Documentation Types

VNAGY requires the following documentation artifacts:

- **Module Documentation** — describes module purpose, inputs, outputs, constraints.
- **Component Documentation** — describes component boundaries and responsibilities.
- **State Documentation** — describes deterministic state transitions.
- **Error Documentation** — describes explicit error types and conditions.
- **Configuration Documentation** — describes all configurable parameters.
- **README.md** — minimal overview of the directory or module.

XII. III. Module Documentation Rules

Each module **MUST** include a `README.md` containing:

- **Purpose** — one sentence.
- **Inputs** — explicit list.
- **Outputs** — explicit list.
- **State Dependencies** — explicit list.
- **Error Conditions** — explicit list.
- **Deterministic Guarantees** — explicit list.

Module documentation **MUST NOT** include:

- narrative explanations
- implementation details
- implicit behavior
- dynamic or runtime-dependent descriptions

XII. IV. State Documentation Rules

State documentation **MUST** include:

- **State Definitions**
- **Allowed Transitions**
- **Transition Preconditions**
- **Transition Postconditions**
- **Error States**

State documentation **MUST NOT** include:

- implicit transitions
- dynamic state mutation
- undocumented fallback behavior

XII. V. Error Documentation Rules

Error documentation **MUST** include:

- **Error Name**
- **Error Condition**
- **Error Severity**
- **Error Recovery Path**
- **Error Determinism Guarantee**

Error documentation **MUST NOT** include:

- ambiguous error descriptions
- catch-all error categories
- implicit error behavior

XII. VI. Configuration Documentation Rules

Configuration documentation **MUST** include:

- **Parameter Name**
- **Parameter Type**
- **Default Value**
- **Allowed Values**
- **Deterministic Effect**
- **Offline-First Behavior**

Configuration documentation **MUST NOT** include:

- hidden parameters
- dynamic defaults
- undocumented side effects

XII. VII. README Rules

Every directory **MUST** contain a `README.md` with:

- **Purpose**
- **Contents**
- **Dependencies**
- **Constraints**

README **MUST NOT** contain:

- narrative text
- implementation details
- personal notes

XII. VIII. Prohibited Documentation Patterns

The following patterns **MUST NOT** appear in VNAGY documentation:

- narrative storytelling
- subjective language
- ambiguous diagrams
- camelCase in headings
- undocumented behavior
- implicit assumptions
- dynamic descriptions
- “TODO” placeholders
- mixed formatting styles
- inconsistent terminology

XIII. VNAGY Security Requirements

All **VNAGY** components MUST follow strict security rules ensuring deterministic, isolated, offline-first, and non-reconstructable behavior. Security is not a feature; it is a structural constraint of the **VNAGY** architecture.

XIII. I. Isolation Requirements

- All modules **MUST** operate in isolated execution contexts.
- No module **MAY** access another module's internal state directly.
- Cross-module communication **MUST** occur only through explicit, documented interfaces.
- Global mutable state **MUST NOT** exist.
    - *"Global mutable state is prohibited."*

XIII. II. Deterministic Execution Requirements

- All **VNAGY** components **MUST** behave deterministically under identical inputs.
- Randomization **MUST NOT** be used in any security-relevant logic.
- Dynamic execution **MUST NOT** be used.
    - *"Dynamic code execution is prohibited."*
- All IO operations **MUST** be explicit and documented.
    - *"All IO operations must be explicit."*

XIII. III. Offline-First Requirements

- All detection, correlation, scoring, and response logic **MUST** execute locally.
- No **VNAGY** module **MAY** depend on cloud connectivity for security decisions.
- Local baselines, thresholds, and scoring profiles **MUST** be stored offline.
- Remote services **MAY NOT** influence operational behavior.

XIII. IV. Supply-Chain Security Requirements

VNAGY includes a full supply-chain security layer (confirmed in your project structure):

> "c**onfig-integrity-shield**", "**dependency-freeze**", "**digital-signatures**", "**hash-verification**", "**ingestion-anti-corruption**", "**offline-build-pipeline**", "**usb-guard**"

Therefore:

- All dependencies **MUST** be frozen and cryptographically verified.
- All modules **MUST** include integrity markers validated at load time.
- All ingestion pipelines **MUST** include anti-corruption guards.
- USB or external ingestion **MUST** be sandboxed and validated.

XIII. V. Non-Reconstructability Requirements

Your Coding Standard explicitly states:

> *"No operational logic is exposed." "Public documents contain exclusively symbolic constructs." "Any component that can be misused or reverse-engineered is not publicly released."*

Therefore:

- Public **VNAGY** documents **MUST NOT** contain operational logic.
- Thresholds, weights, scoring parameters, correlation rules, and runtime behavior **MUST NOT** be published.
- Symbolic descriptions **MUST** be designed so they cannot be transformed into operational behavior.
- Architecture layers **MUST** remain strictly separated (conceptual vs operational).

XIII. VI. Input Validation Requirements

- All inputs **MUST** be validated structurally and semantically.
- Invalid inputs **MUST** trigger explicit errors.
- Silent failure **MUST NOT** occur.
- Input normalization **MUST** be deterministic and reproducible.

XIII. VII. Error Security Requirements

- All errors **MUST** be explicit and deterministic.
- Error messages **MUST NOT** leak internal logic, thresholds, or sensitive state.
- Error recovery paths **MUST** be documented and predictable.
- Catch-all errors **MUST NOT** exist.

XIII. VIII. Logging Requirements

Your Coding Standard states:

> "*Only structural events may be logged: start, stop, fail, recover.*"

Therefore:

- Logs **MUST** contain only structural events.
- Logs **MUST NOT** contain sensitive data, heuristics, or detection logic.
- Debug logging **MUST NOT** exist.
- Logging **MUST** be minimal and deterministic.

XIII. IX. Testing Requirements

- All tests **MUST** be deterministic.
- Randomization **MUST NOT** be used.
- Tests **MUST** run in isolation.
- Every function **MUST** have a unit test.
- Every module **MUST** have an integration test.
    - *"Tests must produce consistent results."*

XIV. VNAGY Code Examples

XIV. I. Module Structure Example (main.vng)

```
module main

import state
import errors
import types

function initialize(input)
    validate input
    state.transition("INIT")
    return types.Result("OK")
end
```

XIV. II. State Definition Example (state.vng)

```
state INIT
state READY
state FAILED

transition INIT → READY
transition READY → FAILED
transition FAILED → READY
```

XIV. III. Error Definition Example (errors.vng)

```
error InvalidStateError
    condition: state not in allowed set
    severity: high
    recovery: reset state to INIT
end

error MissingInputError
    condition: input is null
    severity: medium
    recovery: abort
end
```

XIV. IV. Type Definition Example (`types.vng`)

```
type Result
   field status: string
end

type DeviceState
   field id: string
   field mode: string
end
```

XIV. V. Deterministic Function Example

```
function validate(input)
   if input is null
      raise MissingInputError
   end
end
```

XIV. VI. Configuration Example (`config.vng`)

```
config
   DEFAULT_MODE = "offline"
   MAX_RETRY = 3
   TIMEOUT_MS = 5000
end
```

XIV. VII. Component README Example (`README.md`)

device-detection

Purpose:
  Deterministic offline-first device identification.

Inputs:
  device_id
  profile

Outputs:
  DeviceState

Constraints:
  No network access.
  No dynamic execution.
  No global state.

XV. Appendix

Appendix sekcije **MAY** contain supplemental, non-normative material that supports the **VNAGY** standard. Appendix content **MUST NOT** introduce new requirements, rules, or constraints.

A. Compliance Checklist

This checklist provides a minimal, deterministic, and explicit set of verification points to ensure that all components, documents, and contributions conform to the **VNAGY** Coding Standard. Each item represents a structural requirement. No operational, algorithmic, or implementation-specific criteria are included.

A. 1. Structural Compliance

- Code follows minimal, deterministic, and explicit design principles.
- No hidden behavior, implicit flows, or uncontrolled dependencies.
- Module boundaries are respected; no unauthorized cross-module interactions.

A. 2. Naming & Formatting

- All identifiers follow **VNAGY** naming conventions (variables, constants, functions, types, modules).
- No abbreviations, no ambiguous names, no stylistic deviations.
- Documentation uses the prescribed technical tone and structure.

A. 3. Error & Logging Rules

- All errors are explicit, fail-fast, and structurally predictable.
- Logging is minimal and limited to structural events only.
- No verbose output, debug spam, or implicit logging.

A. 4. Dependency & Security Rules

- External dependencies are minimized and justified.
- No global state, no reflection, no dynamic execution.
- All IO operations are explicit and controlled.

A. 5. Dependency & Security Rules

- External dependencies are minimized and justified.
- No global state, no reflection, no dynamic execution.
- All IO operations are explicit and controlled.

A. 6. Testing Requirements

- Tests are deterministic, isolated, and free of randomization.
- Coverage includes both unit and integration tests.
- No reliance on external or mutable state.

A. 7. Documentation Requirements

- Documentation is minimal, technical, and non-narrative.
- Structure follows **VNAGY** documentation rules.
- No operational details, thresholds, heuristics, or algorithms are included.

A. 8. Licensing & IP Compliance

- Attribution is present and correct.
- No prohibited use (commercial, derivative, reconstructive, or impersonation).
- No removal or modification of licensing terms.
- No exposure of operational logic or internal mechanisms.

B. Glossary

B. 1. Deterministic Execution

A predictable and repeatable execution flow where identical inputs always produce identical outputs. No implicit behavior or hidden state transitions.

B. 2. Explicit State

All state is visible, controlled, and intentionally defined. No hidden, global, or automatically propagated state.

B. 3. Minimal Surface

The smallest possible set of exposed interfaces, behaviors, and interactions. Reduces complexity and limits unintended coupling.

B. 4. Offline-First

A design principle ensuring that all components operate without reliance on external services, networks, or dynamic runtime dependencies.

B. 5. Fail-Fast

A structural rule requiring immediate termination upon detecting invalid input, inconsistent state, or unexpected conditions.

B. 6. Predictable Flow

A strictly ordered and transparent sequence of operations with no dynamic branching, implicit transitions, or runtime-dependent behavior.

B. 7. Non-Operational Content

Information that describes structure, rules, or constraints without revealing algorithms, heuristics, or execution logic.

B. 8. Normative Section

A mandatory part of the document containing rules that must be followed.

B. 9. Non-Normative Section

A contextual or explanatory part of the document that does not impose mandatory requirements.

B. 10. Structural Dependency

A dependency that is explicitly declared, static, and controlled. No dynamic imports, reflection, or runtime discovery.

B. 11. Deterministic Folder Behavior

A rule ensuring that directory structures, file placement, and module boundaries remain fixed and predictable.

B. 12. Explicit Error

An error that is clearly defined, intentionally raised, and structurally documented. No silent failures or implicit recovery.

B. 13. Minimal Documentation

Documentation limited to structural, deterministic, and explicit information without narrative, descriptive, or interpretative content.

C. Terminology Table

Term	Definition
VNAGY	Offline-first security architecture designed for deterministic, isolated, and non-reconstructable operation.
Module	Smallest functional unit in **VNAGY**. Operates in isolation, has explicit inputs/outputs, and no global state.
Component	Logical grouping of modules forming a higher-level capability (e.g., agent, installer).
.vng File	Deterministic **VNAGY** module file containing symbolic structure definitions (types, states, errors).
State	Explicit, deterministic representation of module condition. No implicit transitions allowed.
Transition	Allowed movement between states. **MUST** be explicitly defined.
Error	Deterministic failure condition with explicit severity and recovery path.
Type	Structured data definition used by modules. **MUST** use PascalCase.
Determinism	Guarantee that identical inputs always produce identical outputs. No randomness, no dynamic execution.
Offline-First	**VNAGY** principle requiring all logic to run locally without cloud dependency.
Isolation	Modules cannot access each other's internal state; communication only through explicit interfaces.
Explicit IO	All input/output operations MUST be declared and documented. No hidden IO.
Non-Reconstructability	Public documents contain symbolic constructs only; operational logic is never exposed.
Integrity Marker	Cryptographic signature or hash ensuring module

Term	Definition
	authenticity.
Dependency Freeze	Supply-chain rule requiring all dependencies to be locked and verified.
Config Parameter	Deterministic configuration value with explicit type, default, and allowed range.
Profile	Offline behavioral template used by modules (e.g., detection profile, scoring profile).
Sandbox	Isolated execution environment preventing external influence or contamination.
Structural Event	Minimal logging event: start, stop, fail, recover. No operational details allowed.
Security Boundary	Explicit separation between conceptual and operational layers.

D. Directory Maps

```
vnagy/
├── ui/
│   ├── flutter/       # Mobile UI (Android & iOS)
│   ├── kotlin/        # Mobile service layer
│   └── tauri/         # Desktop UI (Windows, Linux, macOS)
│       ├── webui/     # Desktop frontend (local rendering)
│       └── src-tauri/  # Desktop backend (Rust)
│
├── agent/
│   ├── config/        # Agent configuration
│   ├── integration/    # OS-level integration points
│   ├── src/           # Agent runtime logic
│   └── tests/         # Agent testing suite
│
├── agent-installer/
│   ├── config-generator/   # Installer configuration builder
│   ├── device-detection/   # Platform and hardware detection
│   ├── module-selection/   # Module selection logic
│   └── prof/              # Profiling and environment checks
│
├── core/
│   ├── engine/        # Rust core engine
│   ├── modules/       # Modular system extensions
│   ├── scoring/       # Security scoring layer
│   └── offline-first/  # Local processing and storage
```

```
│
├── processing/
│   ├── heuristics/     # Analytical models (Python)
│   ├── correlation/    # Correlation flows
│   └── pipelines/      # Data processing pipelines
│
├── integration/
│   ├── api/           # Integration interfaces
│   ├── adapters/      # Data transformation
│   └── bridges/       # Cross-layer communication
│
└── installer/
    ├── mobile/        # Mobile build outputs
    └── desktop/       # Desktop build outputs (MSI, DMG, AppImage)
```

E. Example Structures

Non-operational symbolic examples:

- module structure
- state definitions
- error definitions
- type definitions
- config blocks

F. Document Version History

Version	Date	Author	Description
V1.0	20 July 2026	V. Nad	Draft
V1.1	22 July 2026	V. Nad	License & Footer Integrated
V1.2	01 August 2026	V. Nad	Structural & Content Expansion Update

G. Future Extensions (Non-Normative)

This section **MAY** list ideas for future **VNAGY** expansions, npr.:

- additional module types
- extended offline-first profiles
- new deterministic patterns

- expanded documentation templates

H. References (Non-Normative)

Any external materials that inspired or informed the architecture, npr.:

- ISO/IEC standards
- NIST SP 800 series
- RFC documents
- academic papers

Table of Contents

1. Purpose

This document defines the **VNAGY** programming standard.The objective is to ensure minimal, deterministic, explicit, and structurally predictable code across all **VNAGY** modules and components.

2. Core Principles

2.1. Minimalism

Code must contain only what is necessary. No redundant constructs, no decorative abstractions, no “cleverness”.

2.2. Determinism

Execution must be predictable. No hidden side-effects, no implicit state transitions.

2.3. Explicitness

All behavior must be explicit. No magic values, no implicit conversions, no silent fallbacks.

2.4. Isolation

Modules must remain isolated. Cross-module dependencies are prohibited unless strictly required.

2.5. Predictable Flow

Control flow must remain linear and readable. Maximum nesting depth: 2 levels.

3. Naming Conventions

3.1. General

- English only
- Short, precise, descriptive
- No unnecessary abbreviations

3.2. Variables

`lower_snake_case`

3.3. Constants

`UPPER_CASE`

3.4. Functions

`verb_noun`

3.5. Classes / Types

`PascalCase`

3.6. Modules

`lower_snake_case`

4. Error Handling

4.1 Explicit Errors

All errors must be raised explicitly. Silent failure is prohibited.

4.2. Fail-Fast

Errors must be thrown immediately upon detection.

4.3. Predictable Format

Error structures must remain consistent across modules.

5. Logging Standard

5.1. Minimal Logging

Only structural events may be logged:

- start
- stop
- fail
- recover

5.2. No Verbose Output

Debug spam is prohibited.

6. Dependency Rules

6.1. Minimal Dependencies

External libraries must be avoided unless industry-standard.

6.2. Prefer Standard Library

Standard library usage is preferred whenever possible.

6.3. No Utility Abstractions

Libraries that “do everything” are prohibited.

7. Security Rules

7.1. No Global State

Global mutable state is prohibited.

7.2. No Reflection

Reflection is prohibited unless strictly required.

7.3. No Dynamic Execution

Dynamic code execution is prohibited.

7.4. No Implicit IO

All IO operations must be explicit.

8. Testing Standard

8.1. Deterministic Tests

Tests must produce consistent results.

8.2. No Randomization

Randomized behavior is prohibited.

8.3. Isolation

Tests must not depend on external state.

8.4. Coverage

Every function must have a unit test. Every module must have an integration test.

9. Documentation Standard

9.1. Minimal Documentation

Only essential information is documented.

9.2. Technical Tone

No marketing language. No storytelling.

9.3. Structure

- One paragraph per module
- One sentence per function

10. Code Review Rules

10.1. Structural Feedback Only

Subjective comments are prohibited.

10.2. Reject Complexity

Any unnecessary complexity must be removed.

10.3. Focus Areas

- correctness
- clarity
- security

11. VNAGY Architecture Rules

Implementation Exists; Public Access Limited to Non-Operational Content

11.1. Implementation Status

All VNAGY modules (M-Module, C-Pack, L-Pack, S-Pack, X-Pack) **exist in implemented form** within the private VNAGY development environment. The architecture is functional, modular, and internally validated.

11.2. Public Availability

Only **non-operational**, **non-computational**, and **non-reconstructable** components are made publicly available. Public documents contain exclusively:

- symbolic constructs
- conceptual interaction rules
- structural descriptions
- non-algorithmic behavior models
- licensing and IP constraints

No operational logic is exposed.

11.3. Restricted Content

The following implementation details are **not** publicly released:

- detection logic
- thresholds, weights, scoring parameters
- correlation rules
- runtime behavior
- operational semantics
- executable modules
- internal pipelines
- any component that can be misused or reverse-engineered

11.4. Separation of Layers

The conceptual layer (public) and the operational layer (private) remain strictly separated. No leakage of implementation details is permitted.

11.5. Security Constraint

All public VNAGY documents are intentionally designed to be **non-reconstructable**, ensuring that no symbolic description can be transformed into operational behavior.

11.6. Reference to VNAGY Whitepaper

This coding standard aligns with the architectural principles defined in: *VNAGY Secure-First Architecture — Internal Technical Specification v1.0 (Osijek / Bucharest)*. The whitepaper provides the conceptual and symbolic foundation of the system. The operational implementation exists but is not publicly released.

12. Intellectual Property & Licensing

12.1. Ownership

The VNAGY Secure-First Architecture, including all symbolic constructs, interaction rules, structural definitions, and conceptual models, is the exclusive intellectual property of the author.

All rights are reserved.

12.2. Permitted Use

The following actions are permitted:

- reading and personal study
- academic reference
- citation with attribution
- sharing the document in its original, unmodified form

12.3. Prohibited Use

The following actions are strictly prohibited:

- implementation of any construct, rule, or model
- derivation of operational logic from symbolic definitions
- reconstruction of algorithms, thresholds, weights, or detection semantics
- commercial use in any form
- modification, removal, or alteration of licensing terms
- redistribution of modified versions

12.4. Non-Operational Declaration

This document contains no operational details, no thresholds, no weights, no heuristics, no correlation logic, and no algorithmic semantics. All constructs are symbolic and conceptual. The model is non-computational and non-reconstructable by design.

12.5. License

VNAGY Extended License (PSDL-1.3)

VNAGY CC BY-NC 4.0 Extended License

License reference:
https://github.com/VNAGY-Framework/VNAGY/blob/main/LICENSE